

Research Report: Toolkit Audit + Model-Switching for Token/Latency Savings

Issue: #123 | **Date:** 2026-06-10 | **Toolkit version audited:** 5.1.0 (commit 832b1af) **Cycle type:** research only - this report identifies changes; nothing was implemented.

TLDR

The issue asked "where should we build a harness to switch between Opus, Sonnet, and Haiku?" The evidence answers a slightly different question, and that reframe is the headline:

1. **Nobody in the ecosystem routes models with a model.** Across 8 audited toolkits (including the two most-starred Claude Code projects), the dominant pattern is **static model pins in subagent frontmatter** plus, at most, thin deterministic rules (keyword scoring, complexity thresholds). Garry Tan's gstack actually *built* a rule-based model router and then *removed it*. The "harness" we need is one line of configuration per dispatch site, not infrastructure.
2. **Our toolkit pins nothing today.** Every workload - commands, /review's 4 subagents, /index's 5 chunk agents, /execute's parallel agents - inherits the session model (for this user, often Fable 5, the most expensive tier at 10x Haiku's price).
3. **Token-reduction is the bigger speed lever than model-switching** for most of our measured hot spots. The audits supplied a rich menu of proven techniques (progressive disclosure, hard reading budgets, scripts-before-models, paste-don't-read handoffs, diff-scoped gating).
4. **The quality fear is answerable with data.** On single-pass review quality, Haiku 4.5 measured *equal or better* than Sonnet on a 400-PR benchmark. The unmeasured zone is multi-step agentic work - which is exactly where our guardrails (structured output contracts, one-tier-up escalation, a never-route list) apply.
5. **Measured speed ratios** (not vendor claims): Haiku is ~2.1x faster than Sonnet end-to-end on a typical subagent job, and ~4x faster than Opus-with-thinking. Costs run 1 : 3 : 5 : 10 (Haiku : Sonnet : Opus 4.8 : Fable 5). For a subscription user, "cost savings" means **plan-quota headroom**, not dollars.

13 recommendations follow, ranked speed-first. The top five capture most of the win.

How this research was done

- **Internal measurement:** 4 parallel read-only investigators measured every prompt file, inline injection, fan-out, and HTML artifact in the toolkit, plus profiled the 4 sibling projects.
- **External audit:** 17 agents - 4 discovery sweeps, a selection judge, 7 deep audits plus Garry Tan's setup, a completeness critic, and 4 gap-fill verifiers.
- **Evidence hygiene:** all popularity figures re-verified against the live GitHub API on 2026-06-10; every Claude Code mechanism claim verified against current official docs; vendor multipliers ("saves 30-50%", "89% accuracy", "92% reduction") are labeled author-claimed and were *not* used to rank recommendations. Two corrections the verifiers caught: the

community's "Haiku is 15-25x cheaper than Opus" is stale (it referenced deprecated Opus 4.1 pricing; the real ratio vs Opus 4.8 is 5x, ~6.5x after tokenizer differences), and gsd-plugin's claimed MIT license could not be confirmed (GitHub detects no license file).

Part 1: What we measured in our own toolkit

#	Measured fact	Size
M 1	Claude model pins anywhere in the toolkit	Zero (no command frontmatter exists at all; no <code>skill model: field</code> ; no <code>.claude/agents/</code> ; no settings model key)
M 2	Always-on session load (toolkit.md + html-outputs.md + CLAUDE.mds + memory)	~8.5k tokens, every session
M 3	Session-start reads for <code>/explore</code> , <code>/create-plan</code> , <code>/pair-debug</code> (map + lessons index)	~6k tokens; +8.1k if LESSONS-detail.md opens
M 4	<code>/review</code> fan-out input multiplier (up to 4 subagents, each re-ingesting the same ~14.6k-token shared block + re-reading changed files)	~3.5-4x vs single pass (~36-50k vs ~10-14k input tokens)
M 5	<code>/review</code> output duplication (findings stated as subagent reports, combined markdown, then HTML JSON)	~6-12k output tokens per gated run
M 6	<code>/create-plan</code> hand-written HTML (avg of 7 real plans: 11.8KB)	~2.95k output tokens per plan, default-on, on top of the ~2.3k-token markdown plan
M 7	<code>/index</code> chunk agents	Up to 5 x 250k input tokens; output capped at ~2k each; <code>index.md:42</code> claims "Sonnet" but nothing enforces any model
M 8	Sibling installs (trace-annotator 4.3.3, Resume 4.5.1, spearfish 5.0.0, Email 5.0.0)	None has the v5.1.0 render pipeline; Email hand-rendered 26 HTML artifacts in 10 days

Output tokens are the slow kind (generation, not reading), which is why M5 and M6 punch above their token counts on perceived latency.

Part 2: What the ecosystem does (8 toolkits audited)

Toolkit	Domain	Stars (verified)	Routine g rung	Headline lesson
Garry Tan gstack + gbrain	mixed	108.9k / 22.1k	static-pins	Built a rule-based model router, then removed it; pins an exact Haiku version on one narrow classifier job; speed comes from compiled prompts, tiered preambles, caches, and gating
Superpowers (obra)	coding	223.5k	rule-based (prose)	Prose tier rules + escalation ladder; zero static pins; gold standard for lean on-demand skill loading and paste-don't-read subagent handoffs
oh-my-claudecode	coding	36.1k	rule-based	All 19 agents pin models in frontmatter (7 opus / 10 sonnet / 2 haiku); deterministic keyword router on top; "disagree means take the higher tier"

Toolkit	Domain	Stars (verified)	Routing	Headline lesson
wshobson/agents	coding	36.6k	static-pins	The canonical tier map across 192 agents: 54 opus / 62 sonnet / 20 haiku / 49 inherit; debugging is Sonnet, templated docs are Haiku, only security/architecture/synthesis get Opus
RuFlo (ruvnet)	mixed	58.9k	rule-based	Lexical-score + Thompson-bandit router (real code, marketing overstates it); Opus reserved for 2 of dozens of agents; "89% accuracy" is unsubstantiated
gsd-plugin	coding	69	rule-based	Best engineering on our exact pains: single JSON model catalog + zero-dep resolver, "pin down, inherit up" sentinel, failure-tier escalation, context-budget rulebook
ARIS	research (non-coding)	11.9k	rule-based	Routes by ROLE and model FAMILY (executor vs cross-family reviewer), not capability tier; effort dial with explicit token multipliers (lite 0.4x to beast 5-8x)
claude-obsidian	knowledge mgmt (non-coding)	6.5k	static-pins	Pins Sonnet on all 3 agents; speed comes entirely from a 3-layer read cascade (hot.md ~500 tokens, then index, then pages), hard per-query token budgets, and deterministic scripts

Cross-cutting findings:

- **No LLM-classifier routing exists in any community toolkit.** The two real routers (oh-my-claudecode, RuFlo) are pure code - zero latency, zero tokens. Static pins are the dominant, boring, working answer.
- **The tier-to-job consensus is remarkably consistent:** Haiku = search, indexing, docs, templated/structured output. Sonnet = implementation, debugging, review workers. Opus = architecture, security, final synthesis only. Anthropic's own code-review plugin follows the same shape (Haiku for triage/scoring, Sonnet/Opus for detection).
- **The non-coding toolkits get their speed from caps, caching, and deterministic scripts rather than routing.** That is a valid result, not a gap - and it matches where our own evidence points.
- **Several toolkits independently converged on patterns we already use** (emit JSON + script stamps boilerplate, index-then-detail reads), which validates the toolkit's direction and makes the remaining gaps (no pins, no budgets, no gating) look like the natural next step.

Part 3: What Claude Code natively supports (the mechanism reference)

Verified against current official docs (June 2026). The workload distinction decides everything:

Mechanism	Covers	Verdict for us
model: frontmatter in .claude/agents/*.md	dispatched subagents	The durable pin. Values: haiku/sonnet/opus/fable/full-ID/inherit (default inherit)

Mechanism	Covers	Verdict for us
Agent tool per-invocation model parameter	one dispatch	The lightest pin - a prompt-level instruction at the dispatch site. Honored unless the env var overrides
<code>model:</code> frontmatter on skills/commands	main-loop commands	Avoid mid-session: each model keeps its own prompt cache, so a per-command override forces an uncached re-read of the whole conversation, twice (override + revert). The community has not adopted main-loop pins; they route via subagents instead
<code>context:</code> fork + <code>agent:</code> Explore on a skill	runs a skill as a Haiku read-only subagent that skips CLAUDE.md	Interesting future packaging for read-only commands; not needed now
<code>CLAUDE_CODE_SUBAGENT_MODE</code> env var	ALL subagents	Footgun - it also downgrades the built-in Plan agent. Do not use
PreToolUse hook with <code>updatedInput</code>	central enforcement	Officially supported for rewriting tool input; oh-my-claudecode ships deny+retry instead (battle-tested fallback). Over-engineering for our 6 dispatch sites today
<code>effort</code> frontmatter on subagents	speed knob without model change	Free on subagents (they build their own cache); cache-busting on main-loop skills
Fast mode (/fast)	Opus main loop	Requires usage credits enabled (bills outside the subscription from the first token). Not a free lever for this user

The key implication: route via subagents, never via main-loop model switches. Subagent pins have zero cache penalty; main-loop switches cost one slow uncached turn each way.

The measured numbers behind the recommendation (gap-fill verification, sources in appendix):

- Cost per MTok: Haiku \$1/\$5, Sonnet \$3/\$15, Opus 4.8 \$5/\$25, Fable 5 \$10/\$50 - exactly 1 : 3 : 5 : 10.
- End-to-end on a typical ~2k-output-token subagent job: Haiku ~24s, Sonnet ~49s, Opus-with-thinking ~94s. The Opus-tier penalty is not serving speed (its raw throughput beats Sonnet); it is the thinking phase before the first output token.
- For a subscription user: within plan limits there is no marginal dollar cost. Down-tiering buys **rate-limit headroom** ("Opus costs several times more per turn than Sonnet, and Sonnet more than Haiku" - official support docs) and **latency**. Dollars only appear if usage credits are enabled and limits exceeded.

The quality evidence (the user's stated fear):

- Qodo's 400-real-PR benchmark: Haiku 4.5 *beats* Sonnet on single-pass review quality (58% win rate in thinking mode, 7.29 vs 6.60 quality score). Read-and-summarize evals agree.

- The unmeasured zone is **multi-step agentic work** (navigating a repo, chasing cross-file context). The few documented failures (hallucinated function output, "unacceptable basic reasoning", a real bug dropped by a Haiku confidence-scoring stage in Anthropic's own pipeline) all sit there.
- Anthropic's own code-review plugin keeps Sonnet/Opus on *detection* and uses Haiku only for triage and scoring - the model assignment we mirror below.
- One published operational heuristic: if a cheap-tier agent's output needs higher-tier correction more than ~20% of the time, the re-prompting cost negates the price advantage - revoke the pin.

Part 4: Recommendations, ranked by speed impact

Each item: what, evidence, lever, and ratings (Speed / Cost = quota headroom / Effort / Risk).

R1. Migrate plan HTML onto render-html.js (add a plan shell)

The model hand-writes ~11.8KB of HTML per plan (~2.95k slow output tokens, default-on). Add `plan-shell.html` + a stable-name output mode to the helper. Verified scope: three edit sites inside `render-html.js` (lines 28, 64, 74), a new output mode (the helper currently hardcodes timestamped never-overwrite names in `artifacts/html/`; plan HTML needs stable identity-keyed names in `plans/` with `overwrite-on-replan`), `create-plan.md` lines 161-202 rewritten to emit JSON, `rules/doc` updates - ~9 files. Installers need zero edits (shells are copied by glob). **Lever:** token-reduction (scripted generation). **Speed:** High - the known per-plan sink. **Cost:** Medium. **Effort:** Low-Medium. **Risk:** Very low (malformed JSON dies before any file write).

R2. Pin models on the subagent fleets, with guardrails (the "harness" answer)

The full design is Part 5 below. Short version: `/index` chunk agents get Haiku via the Agent-tool model parameter (strict structured output, the safest candidate, also fixes the false "Sonnet" claim at `index.md:42`); `/review` specialist subagents get Sonnet; `/execute` agents and all main-loop judgment work stay on the session model. Start with prompt-level parameters (one line per dispatch site, zero new files); graduate to `.claude/agents/` definitions only if pins multiply. **Lever:** model-switching. **Speed:** High on `/review` and `/index` runs (Sonnet workers return ~2x faster than Opus-with-thinking; Haiku ~4x). **Cost:** High (the fan-outs are the token bulk; 3-10x cheaper per token). **Effort:** Low. **Risk:** Medium, bounded by guardrails (Part 5).

R3. Restructure `/review` output: findings as structured data, once

Today findings are generated up to three times (subagent prose, combined markdown, HTML JSON; M5: 6-12k output tokens). Adopt the convergent pattern from `gstack` and `ARIS`: subagents emit one JSON finding per line (or the literal "NO FINDINGS") with a dedup key; the orchestrator dedups mechanically for free, then derives both the markdown report and the HTML payload from that single structure. **Lever:** token-reduction. **Speed:** High (output-side, the slowest tokens). **Cost:** Medium-High. **Effort:** Medium. **Risk:** Low-Medium (report prose quality depends on the structure carrying enough detail).

R4. Diff-scoped review gating + zero-token skill detection

gstack skips ALL review specialists when the diff is under 50 lines and dispatches conditionally on scope signals; RuFlo uses a tiny regex pre-router script to pick agents before any model reasoning. Our /review currently uses model judgment to pick skills and always fans out. A small-diff skip plus a zero-dep detection script makes the *most common case* (small change) fast and nearly free. **Lever:** token-reduction. **Speed:** High for small changes (the majority of runs). **Effort:** Low-Medium. **Risk:** Low (security/full reviews can be marked never-gate, copying gstack's [NEVER_GATE] insurance).

R5. Trim what each review subagent carries and reads

Each of up to 4 subagents re-ingests the same ~14.6k-token shared block and re-reads the changed files (M4). Three convergent fixes from the audits: paste-don't-read handoffs (controller reads files once, pastes relevant excerpts - Superpowers), point-at-paths for big files instead of inlining (gsd-plugin's context-budget rulebook), and trimmed per-domain checklists instead of full SKILL.md content. **Lever:** token-reduction. **Speed:** Medium (parallelism masks some wall-clock; per-subagent processing still shrinks). **Cost:** High. **Effort:** Medium. **Risk:** Medium - needs a before/after quality check; pairs naturally with R2's escalation guardrail.

R6. Add reading budgets and output caps to subagent prompts

claude-obsidian declares per-depth token budgets ("Quick never opens individual pages", "Standard reads 3-5 pages max", "stop reading once you have sufficient material"); oh-my-claudecode caps exploration by tier (5/20/unbounded files) and bans full reads of files over 500 lines without an outline pass. Our review/index subagents read freely. Pure prompt change. **Lever:** token-reduction. **Speed:** Medium (bounds the tail latency of slow runs). **Effort:** Low. **Risk:** Low.

R7. Session-init script: one call instead of N reads

/explore, /create-plan, and /pair-debug each start with multiple sequential reads (map, lessons, plans). gsd-plugin aggregates all session context into a single scripted JSON query. A generate-index.js-style "session-init" script emitting one compact JSON (map summary + lessons index + plan statuses) replaces several tool roundtrips at the start of every command. **Lever:** token-reduction. **Speed:** Medium (start-of-command latency, felt every cycle). **Effort:** Low-Medium. **Risk:** Low.

R8. Upgrade the four sibling installs to v5.1.0

None has the render pipeline; Email hand-rendered 26 HTML artifacts in 10 days at model speed. Upgrade order and per-project cautions in Part 6. **Lever:** token-reduction delivered by version catch-up. **Speed:** High for Email and spearfish. **Effort:** Low (run installer + cautions). **Risk:** Low-Medium (two specific cautions, Part 6).

R9. Trim the always-on session load (~8.5k tokens)

toolkit.md (~4.2k) is mostly reference tables consulted occasionally; html-outputs.md (~3k) loads even in sessions that never render HTML. The ecosystem's converged answer: a short always-loaded core plus on-demand reference files, with hard size budgets (wshobson caps context files at 150 lines, CI-enforced; Superpowers budgets words by load frequency; official

docs recommend CLAUDE.md under 200 lines). Same split our LESSONS index/detail already proved. **Lever:** token-reduction. **Speed:** Low-Medium per session, but every session, everywhere, plus every subagent that re-reads rules. **Effort:** Medium. **Risk:** Low-Medium (needs reliable pointers so reference content is opened when needed).

R10. Per-cycle telemetry + verdict sidecar (makes quality drops visible)

oh-my-claudecode persists per-session JSON (tokens per agent, est. cost, slowest tools); ARIS pairs every review with a tiny verdict JSON (PASS/WARN/FAIL...) that scripts can gate on. A small sidecar written by /review and /document would (a) reveal which subagent or render burns the time, making R2's pins tunable with evidence, and (b) give /review-full a deterministic go/no-go input. This is also the instrument for the user's quality fear: it is how a Haiku/Sonnet pin gets audited instead of trusted. **Lever:** enabler for both levers. **Speed:** Indirect. **Effort:** Medium. **Risk:** Low.

R11. Effort dial on heavy commands

ARIS's shared effort contract (lite ~0.4x to beast ~5-8x tokens, per-skill knob tables, hard invariants that never scale down, and a transparency banner). A "-- effort: lite" on /review and /explore that cuts subagent counts and rounds would give a predictable fast path for routine work. **Lever:** token-reduction. **Speed:** Medium when used. **Effort:** Medium. **Risk:** Low (invariants like report-only and severity schema never scale down).

R12. Skill-description and prompt hygiene (free wins)

Superpowers' tested convention: descriptions state only *when* to use ("Use when..."), never summarize the workflow - their A/B testing showed workflow-summarizing descriptions cause the model to skip reading the body. Plus: remove the dead-weight internal fan-out instructions from review skills running under the orchestrator (subagents cannot spawn agents), and fix index.md:42's unenforced "Sonnet" claim as part of R2. **Lever:** token-reduction + correctness. **Speed:** Low individually. **Effort:** Low. **Risk:** None.

R13. Watch-list (documented, not recommended now)

- **Hot-cache hook** (claude-obsidian): a Stop-hook-maintained ~500-word summary injected at SessionStart and re-injected PostCompact. Attractive, but adds hook machinery; revisit after R9.
- **Splitting /explore's two modes** (~4.6k tokens loaded, one mode runs): fold into R9's restructure if it happens.
- **Debate transcript caps** (/ask-*): wall-clock is dominated by external API latency; quality is the product. Lower priority.
- **TDD for prompts** (Superpowers: pressure-test a skill edit with a subagent before merging): philosophically aligned with our lessons loop; consider per-edit, not as infra.
- **context: fork packaging** for read-only commands: revisit when Claude Code plugin/agent conventions settle.

Part 5: The model-routing design (R2 in full)

Tier map per workload:

Workload	Today	Pin to	Why
/index chunk agents (up to 5)	inherits session model	haiku	Strict output contract (structured module blocks, <=2k tokens, no commentary); matches Anthropic's own Explore-agent default; mechanical summarization is the measured Haiku sweet spot. Also fixes the false "Sonnet" message at index.md:42
/review specialist subagents (up to 4)	inherits	sonnet	Review workers navigate the repo (the unmeasured agentic zone), so not Haiku; Sonnet matches Anthropic's own code-review plugin and the wshobson consensus. ~2x faster and 40-70% cheaper than the Opus/Fable session model
Review skills' internal fan-outs (direct invocation)	inherits	sonnet	Same job shape as above
/index synthesis, /review orchestration + final report	inherits	inherit (no pin)	Judgment and synthesis stay on the session model - "pin down, inherit up" (gsd-plugin sentinel pattern: omit the parameter entirely rather than pinning a top-tier ID)
/execute parallel agents	inherits	inherit	They edit real code. Excluded on principle
Main-loop commands (/explore, /create-plan, /document, /peer-review...)	session model	no pin	Main-loop model switches bust the prompt cache mid-session; the entire ecosystem routes via subagents instead
Debate scripts (ask-gpt/ask-gemini)	pinned external models	unchanged	Already a complete, configurable pinning mechanism

Mechanism choice: start with the **Agent-tool model parameter, written into the dispatch instructions** of review.md and index.md ("spawn with model: haiku"). One line per site, zero new files, zero installer changes, trivially reversible. Graduate to `.claude/agents/` definitions (frontmatter pins) only if pin sites multiply or drift becomes a problem. Do NOT adopt: CLAUDE_CODE_SUBAGENT_MODEL (downgrades the built-in Plan agent), main-loop skill frontmatter pins (cache-busting), hook-based enforcement (over-engineering at our scale - 6 dispatch sites).

Guardrails (in priority order):

- Structured-output contracts** - pinned subagents must emit schema-shaped output (JSON findings / module blocks / "NO FINDINGS"). Malformed or thin output is *visible*, not silent. (R3 makes this systematic.)
- One-tier-up escalation** - if the orchestrator judges a pinned subagent's output weak or malformed, re-spawn that one subagent one tier up, once (gsd-plugin's failure-tier escalation; oh-my-claudecode's "on disagreement take the higher tier"). Bounded worst case: one extra spawn.

- 3. **The 20% rule** - if cheap-tier output needs correction more than ~20% of the time, revoke that pin (published operational heuristic). R10's telemetry sidecar is how this gets measured rather than felt.
- 4. **Rollback is one line** - delete the model parameter from the dispatch instruction; everything reverts to inherit. No state, no migration.
- 5. **Never-route list** - code-editing agents, main-loop judgment commands, debate content. Written into the design so future cycles do not re-litigate it.

Part 6: Per-project notes (the 4 sibling installs)

Email (5.0.0, active, data-heavy): Biggest render-pipeline beneficiary - 26 hand-rendered HTML artifacts in 10 days, several over 40KB of model-written markup that v5.1.0 stamps from JSON instead. Second win: R2's Haiku-pinned /index chunks matter most here (608 JSON data files bloat any scan); also ensure data/ exclusion when regenerating its map. Upgrade first.

spearfish (5.0.0, most active): Upgrade caution - commit 2bda751 (Jun 8) locally edited toolkit-managed files (toolkit.md line 198 plus frontmatter added to all 14 command files). A reinstall overwrites commands/ and rules/: diff and consciously merge or accept the loss before running setup. It is also the only project showing context pressure (autocompact override), and its 99-file markdown corpus makes /review-copy and /index expensive - R2, R5, and R6 land hardest here. Upgrade second.

Resume (4.5.1, dormant since May 17): Large custom surface that must survive upgrade: commands/res/ (11 files), skills res-apply and res-audit, shared/bullet-quality.md, custom settings additions. The installer skips files it does not own, but verify on a dry pass given two majors of drift. Smallest speed gain from R1-R6 (tiny repo); the custom res-apply orchestrator could adopt R2's Sonnet pins for its own sub-tasks. Upgrade when revived.

trace-annotator (4.3.3, dormant since May 12): Oldest install, pre-HTML entirely; benefits only if revived. During the same pass, remove the broad `gh auth permission from its settings` (known token-leak risk; should be `gh auth status`). Upgrade when revived.

Common to all four: every recommendation that lands in toolkit command files (R2-R7, R9, R12) propagates automatically at their next setup run - one more reason to keep the upgrade path smooth.

Part 7: Proposed issue candidates

The report does not pre-commit a count; these are the natural bundles if everything moves forward. The user decides which become issues.

Candidate	Covers	Size feel
A. Plan HTML to render-html.js	R1	Small, well-scoped, verified
B. Model pins + guardrails on subagent dispatch sites	R2, index.md fix from R12	Small prompt edits + design doc

Candidate	Covers	Size feel
C. Review pipeline economy	R3 + R4 + R5 + R6 (JSON contract, dedup, gating, budgets)	The big one; could split into C1 (output/JSON) and C2 (gating/budgets)
D. Prompt-weight reduction	R9 + R7 + R12 (rules restructure, session-init script, description hygiene)	Medium
E. Sibling upgrade runbook	R8 + Part 6 cautions	Small, mostly operational
F. Telemetry + verdict sidecar	R10 (enables the 20% rule and go/no-go gating)	Medium
Watch-list (R11, R13)	documented here, no issue yet	-

Appendix: key sources

Official docs (verified live 2026-06-10): sub-agents model selection (code.claude.com/docs/en/sub-agents), skills frontmatter (docs/en/skills), model-config + effort + fast mode (docs/en/model-config, docs/en/fast-mode), prompt caching (docs/en/prompt-caching), costs + subscription semantics (docs/en/costs; support.claude.com articles 14552983, 12429409, 11145838), hooks `updatedInput` (docs/en/hooks), pricing (platform.claude.com/docs/en/about-claude/pricing).

Audited repos: github.com/garrytan/gstack + [/gbrain](https://github.com/gbrain), [obra/superpowers](https://github.com/obra/superpowers), [Yeachan-Heo/oh-my-claudecode](https://github.com/Yeachan-Heo/oh-my-claudecode), [wshobson/agents](https://github.com/wshobson/agents), [ruvnet/ruflo](https://github.com/ruvnet/ruflo), [jnuyens/gsd-plugin](https://github.com/jnuyens/gsd-plugin), [wanshuiyin/Auto-claude-code-research-in-sleep](https://github.com/wanshuiyin/Auto-claude-code-research-in-sleep), [AgriciDaniel/claude-obsidian](https://github.com/AgriciDaniel/claude-obsidian).

Measurements: artificialanalysis.ai per-model pages (Haiku 4.5, Sonnet 4.6, Opus 4.8, Fable 5); Qodo 400-PR Haiku-vs-Sonnet review benchmark (qodo.ai blog); Anthropic code-review plugin model assignments (github.com/anthropics/claude-code and [/claude-plugins-official](https://github.com/anthropics/claude-plugins-official)); Augment Code 20%-correction heuristic (augmentcode.com/guides/ai-model-routing-guide).

Evidence caveats carried from the verifiers: all self-reported toolkit multipliers labeled author-claimed; gsd-plugin license unconfirmed; gbrain production-scale numbers self-reported; multi-step agentic quality of Haiku-class models remains unmeasured anywhere - guardrails, not benchmarks, are the current safety net there.